

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3623 - DATABASE PROGRAMMING 2025/2026 - SEM 1

Lab Assignment 1 (5%) - Indexes

Name	:	Goe Jie Ying
Matric Id	:	A23CS0224

Name	:	Lam Yoke Yu
Matric Id	:	A23CS0233

Name	:	Tan Yi Ya
Matric Id	:	A23CS0187

Name	:	Teh Ru Qian
Matric Id	:	A23CS0191

Definition: Unique Index vs Primary Key

A **Unique Index** is a database index that ensures **no two rows** in a table have the **same value** in the indexed columns.

Aspect	Unique Index	Primary Key
Null Values	Allows 1 or more NULL values (depending on DBMS)	Does not allow NULL values
Number per Table	A table can have many unique indexes	Only 1 primary key per table
Constraint Type	Unique constraint, not necessarily an identifier	Main identifier of the record

Advantages

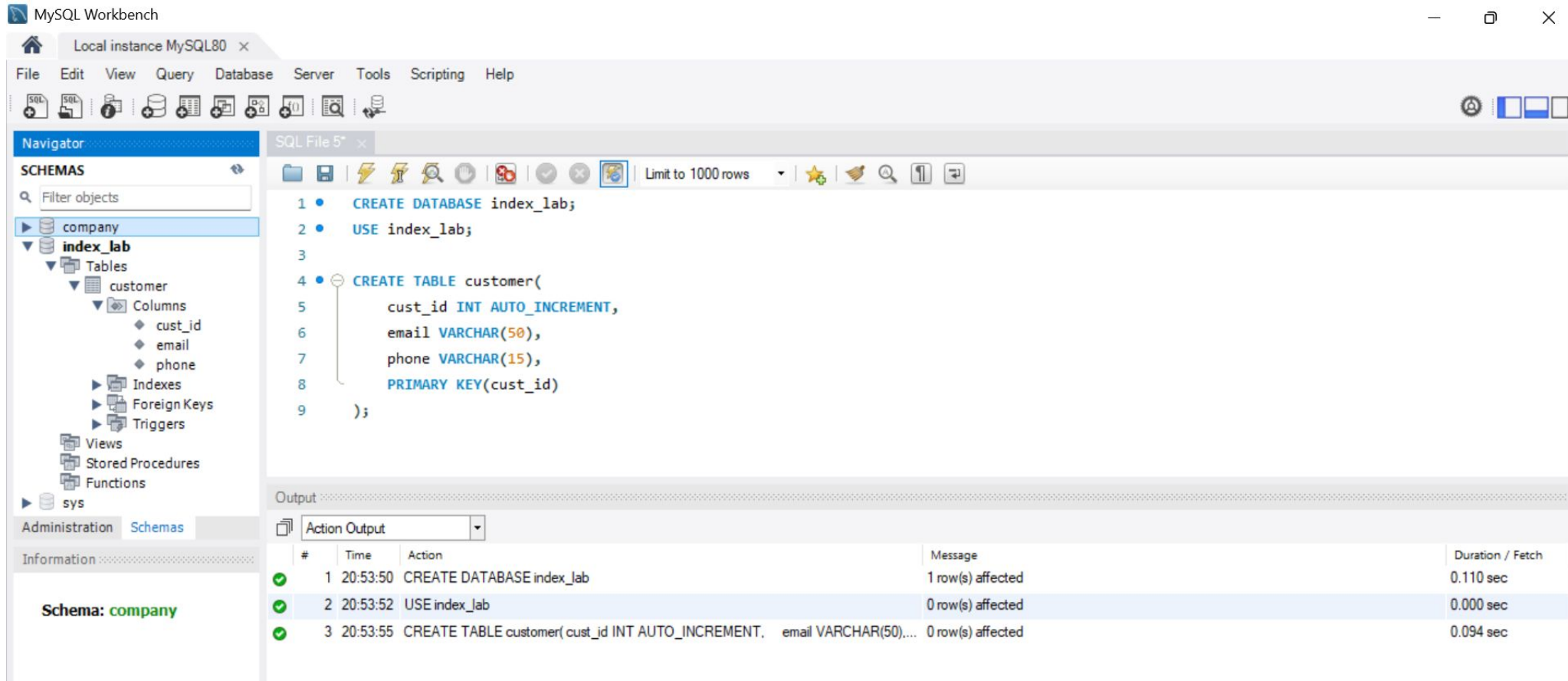
1. **Prevents duplicate data:** maintains data integrity automatically
2. **Improves query performance:** speeds up searches and lookups on indexed columns
3. **Supports data consistency:** ensures unique attributes (email/IC number) remain distinct.

Disadvantages

1. **Slower data modification:** insert and update operations take longer due to uniqueness checks
2. **Consumes extra storage:** indexes require additional disk space
3. **Increases maintenance overhead:** managing and updating indexes adds complexity during database operations.

SQL Command to Create Customer Table

This creates a new database *index_lab*, switches to it and defines a *customer* table. The *cust_id* is the Primary Key, ensuring each customer has a unique (non-null identifier).



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'company' selected. The main editor shows the following SQL commands:

```

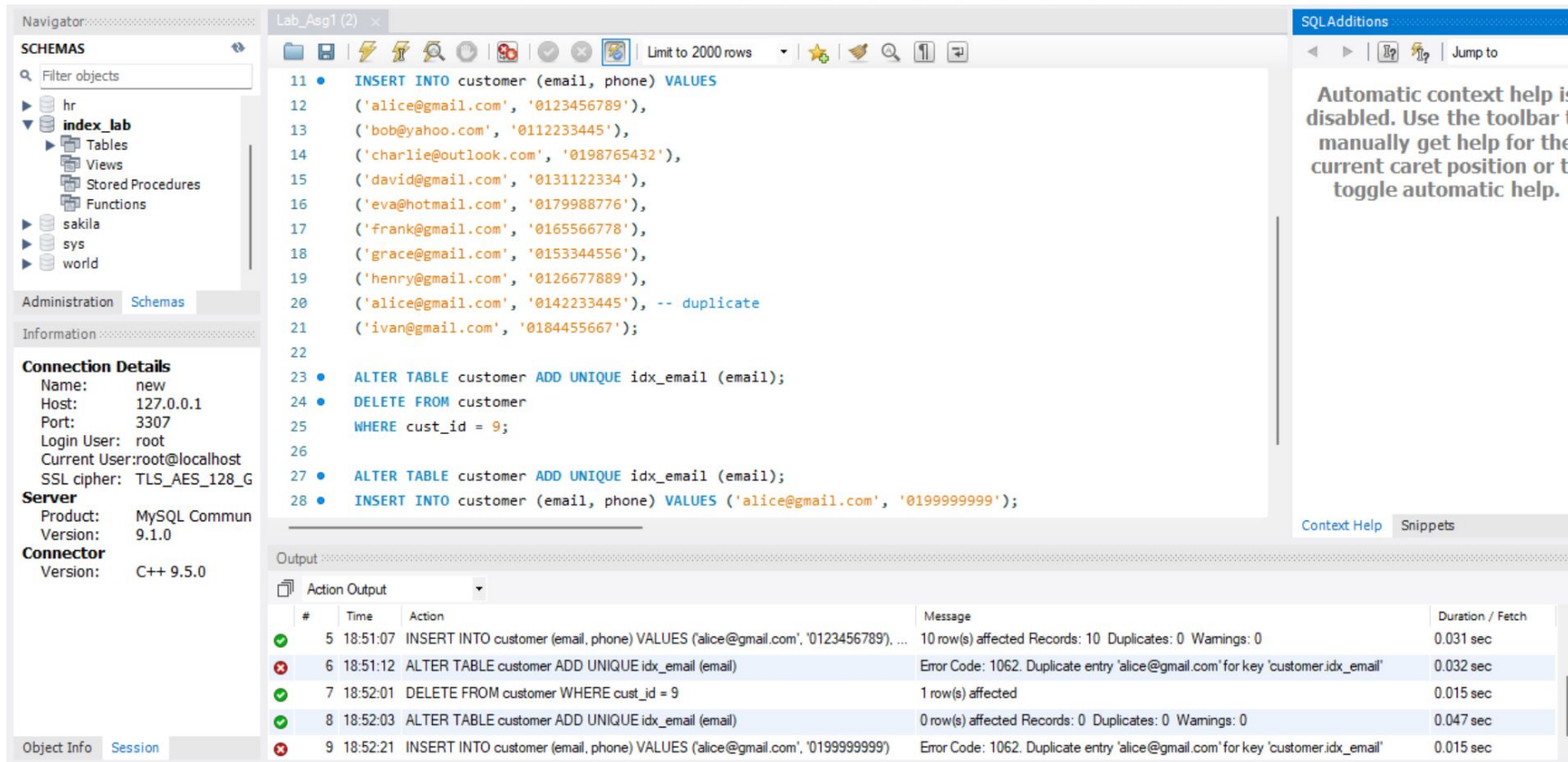
1 • CREATE DATABASE index_lab;
2 • USE index_lab;
3
4 • CREATE TABLE customer(
5     cust_id INT AUTO_INCREMENT,
6     email VARCHAR(50),
7     phone VARCHAR(15),
8     PRIMARY KEY(cust_id)
9 );
  
```

The 'Output' tab at the bottom shows the execution results:

#	Time	Action	Message	Duration / Fetch
✓ 1	20:53:50	CREATE DATABASE index_lab	1 row(s) affected	0.110 sec
✓ 2	20:53:52	USE index_lab	0 row(s) affected	0.000 sec
✓ 3	20:53:55	CREATE TABLE customer(cust_id INT AUTO_INCREMENT, email VARCHAR(50),...	0 row(s) affected	0.094 sec

Inserting Duplicate Records Before and After Adding Unique Index

Before adding Unique Index on email, query that adds duplicate email is executed successfully as shown in Action Output 5. The query to add unique index cannot be executed because a duplicate record on email already exists, as shown in Output 6. An attempt to add duplicate email record after adding Unique Index results in failure, as shown in Output 9.



The screenshot displays the MySQL Workbench interface with the following components:

- Navigator:** Shows the database schema structure including 'hr', 'index_lab', 'sakila', 'sys', and 'world'.
- Connection Details:**
 - Name: new
 - Host: 127.0.0.1
 - Port: 3307
 - Login User: root
 - Current User: root@localhost
 - SSL cipher: TLS_AES_128_G
- Server:**
 - Product: MySQL Commun
 - Version: 9.1.0
- Connector:**
 - Version: C++ 9.5.0
- SQL Editor:** Contains the following queries:


```

11 • INSERT INTO customer (email, phone) VALUES
12 ('alice@gmail.com', '0123456789'),
13 ('bob@yahoo.com', '0112233445'),
14 ('charlie@outlook.com', '0198765432'),
15 ('david@gmail.com', '0131122334'),
16 ('eva@hotmail.com', '0179988776'),
17 ('frank@gmail.com', '0165566778'),
18 ('grace@gmail.com', '0153344556'),
19 ('henry@gmail.com', '0126677889'),
20 ('alice@gmail.com', '0142233445'), -- duplicate
21 ('ivan@gmail.com', '0184455667');
22
23 • ALTER TABLE customer ADD UNIQUE idx_email (email);
24 • DELETE FROM customer
25 WHERE cust_id = 9;
26
27 • ALTER TABLE customer ADD UNIQUE idx_email (email);
28 • INSERT INTO customer (email, phone) VALUES ('alice@gmail.com', '0199999999');
      
```
- Output:** A table showing the execution results of the queries:

#	Time	Action	Message	Duration / Fetch
5	18:51:07	INSERT INTO customer (email, phone) VALUES ('alice@gmail.com', '0123456789'), ...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.031 sec
6	18:51:12	ALTER TABLE customer ADD UNIQUE idx_email (email)	Error Code: 1062. Duplicate entry 'alice@gmail.com' for key 'customer.idx_email'	0.032 sec
7	18:52:01	DELETE FROM customer WHERE cust_id = 9	1 row(s) affected	0.015 sec
8	18:52:03	ALTER TABLE customer ADD UNIQUE idx_email (email)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
9	18:52:21	INSERT INTO customer (email, phone) VALUES ('alice@gmail.com', '0199999999')	Error Code: 1062. Duplicate entry 'alice@gmail.com' for key 'customer.idx_email'	0.015 sec

Use SHOW INDEX FROM customer

- The table shows the indexes from the customer table.
- cust_id is the primary key, while email is a unique key named idx_email.
- There are 9 distinct rows of unique values.
- In the Index_type column, BTREE indicates that MySQL uses a B-tree structure to perform lookups.

```
32 • SHOW INDEX FROM customer;
```

```
33
```

Result Grid Filter Rows: Export: Wrap Cell Content:														
Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
customer	0	PRIMARY	1	cust_id	A	9	NULL	NULL		BTREE			YES	NULL
customer	0	idx_email	1	email	A	9	NULL	NULL	YES	BTREE			YES	NULL

How a Unique Index Improves Lookup Performance

- A unique index uses a sorted structure, allowing the database to quickly locate the position of the index and find records faster.
- When searching for an email, instead of checking each row one by one, it directly jumps to the position of that email in the index, saving a lot of time and improving lookup performance.
- In addition, unique indexes prevent duplicate entries, keeping the data clean and maintaining its integrity.

Query Performance: with and without index

Statement “EXPLAIN SELECT * FROM customer WHERE email = 'henry@gmail.com' ;” is used to check the performance.

Without index

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	customer	NULL	ALL	NULL	NULL	NULL	NULL	9	11.11	Using where

The entire table is scanned regardless to find all values that matches the condition.

With index

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
►	1	SIMPLE	customer	NULL	const	idx_email	idx_email	203	const	1	100.00	NULL

The query would use the index to locate the matching row.

The number of scanned row is significantly less with the UNIQUE index.

When is a unique index not appropriate

- **When the column can have duplicate values**

UNIQUE index ensures that a column does not have duplicate values, where each values are unique.

- **When the column often changes**

When a column is updated, the database must check every row to ensure that the the updated value is unique. This may be time consuming and may negatively affect performance.

- **When uniqueness is not logically required**

Even if the values are unique, it may not be necessary to use unique index if it is not logically required.